

Recipe Page

Ignition Perspective build instruction for the two-tank batch-mixing station

Active and staged recipe side by side, per-row change highlighting, a live validity indicator, a Download button that runs the PLC's validation handshake and confirms the result, and a Copy button that discards pending edits.

Design decision

The PLC owns the recipe. Ignition never writes the active block DS20--DS24; it writes only the staging block DS70--DS74 and then raises one request bit, C30. The PLC validates all four values together and copies the whole block in a single scan, or copies nothing. Editing a staged value is therefore always safe – nothing reaches the process until Download. The page is built the same way as the Alarm page: one reusable row view, `RecipeRow`, repeated five times by a Flex Repeater, each copy receiving the *names* of its two tags as parameters. Every component sits directly in a view's `root`; no container is nested inside another, which keeps the Designer's drag-and-drop out of the way entirely.

1 Requirements and PLC Handshake

The Overview shows the temperature setpoint and mixing time, but only as read-only text. The Recipe page lets an operator:

- see the recipe the PLC is running now (**active**) next to the one being prepared (**staged**);
- edit the staged values, with each row that differs from the active value highlighted **amber**;
- see at a glance whether all staged values are inside the PLC's limits;
- send the staged recipe to the PLC and be told whether it was accepted;
- throw away pending edits by copying the active recipe back into staging.

1.1 What the PLC does

Staged	Active	Meaning	Valid range
DS70	DS20	Tank A fill target, %	1–100
DS71	DS21	Tank B fill target, %	1–100
DS72	DS22	Mix time, seconds	1–600
DS73	DS23	Temperature setpoint, °F	70–200
DS74	DS24	Recipe number	not checked

SBR-CMD – validation and download, as implemented in the PLC

```

(DS70>0) (DS70<101) (DS71>0) (DS71<101)
(DS72>0) (DS72<601) (DS73>69) (DS73<201) OUT C112 ; RECIPE_VALID

C930 C112 (DS10=0) COPY DS70..DS74 to DS20..DS24 (block copy)
C930 C112 (DS10=0) OUT C31 ; DOWNLOAD_ACCEPTED
{ C930 /C112 }
{ C930 !(DS10=0) } OUT C32 ; DOWNLOAD_REJECTED

```

C930 is the one-scan partner of C30, generated in SBR-0S. The PLC therefore reacts to the *rising edge* of C30, and Ignition must return C30 to 0 before a second download can register.

Why the page does not watch C31 and C32

C31 and C32 are true for exactly one PLC scan. Ignition polls its tags on a fixed rate, far slower than a scan, so it will almost never see either bit. The page confirms a download a different way: after the request, it reads the active and staged blocks back and compares them. Equal means accepted; different means rejected. This works for every rejection path – an out-of-range value (which also seals alarm 10) and a download attempted mid-batch (which raises no alarm at all).

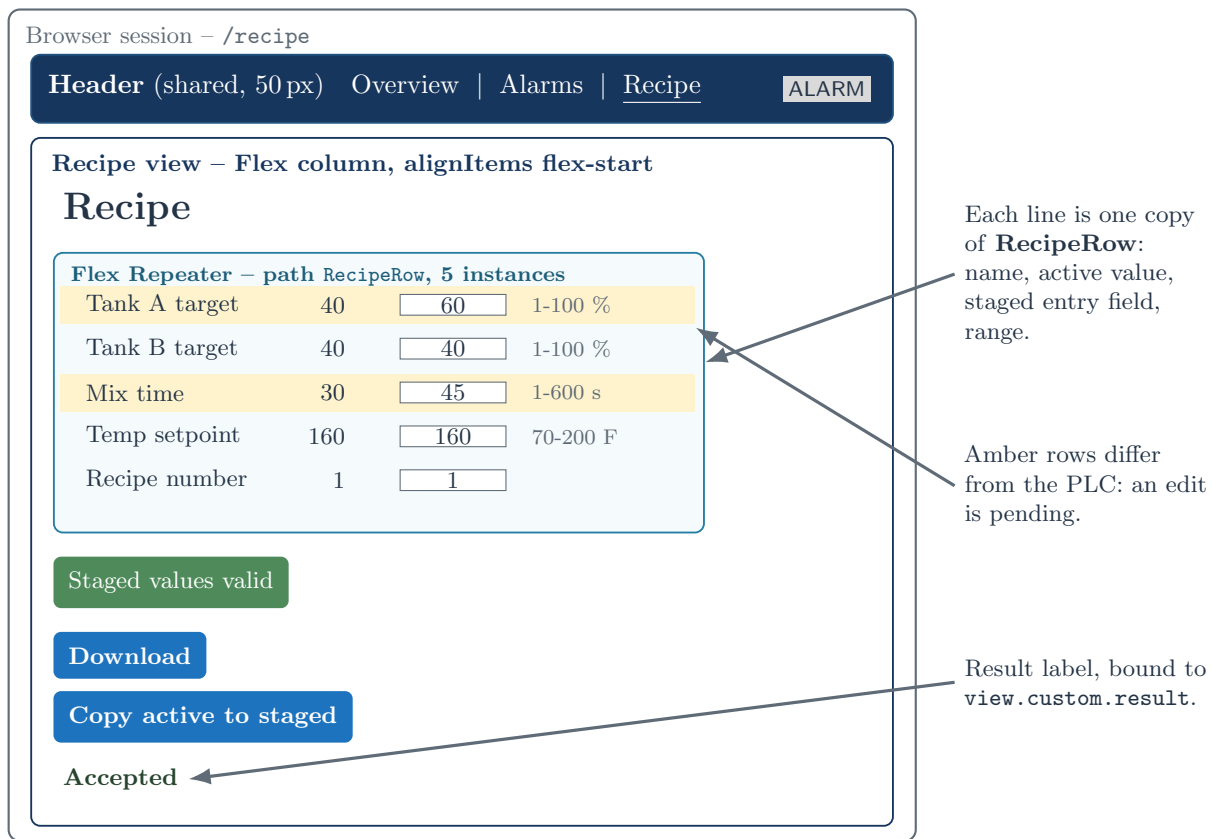
1.2 Tags used by the page

All tags sit at the root of the default provider with flat, underscore-separated names. Confirm each spelling with **right-click** → **Copy Path** in the Tag Browser before typing it into a binding or script.

Tag	CLICK	Modbus	R/W	Used for
Recipe_TargetA ... Recipe_Number	DS20--DS24	400020--24	R	Active column
Staging_TargetA ... Staging_Number	DS70--DS74	400070--74	W	Staged entry fields
Status_RecipeValid	C112	16496	R	Validity indicator
Commands_RecipeDownload	C30	16414	W	Download request
State	DS10	400010	R	Enables Download only when idle

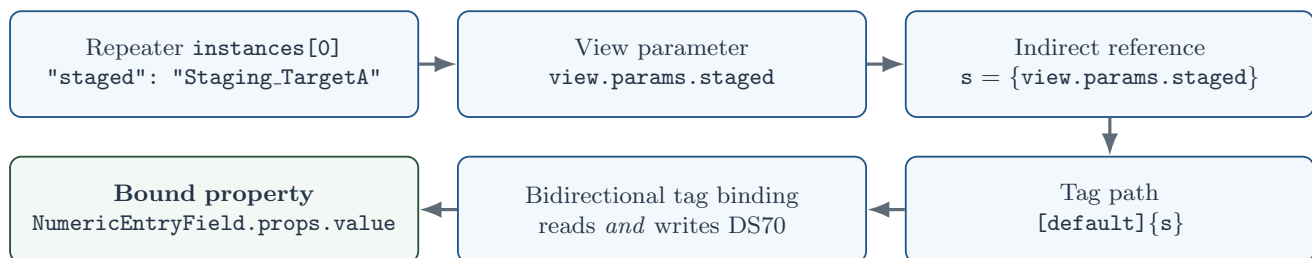
The five names in each group follow the same suffixes: **TargetA**, **TargetB**, **MixTime**, **TempSP**, **Number**. The staging tags and **Commands_RecipeDownload** must **not** be marked Read Only in the tag provider; every other tag on the page should be.

2 Page Structure



View	Page URL	Purpose
Recipe	/recipe	Recipe page. Title, the Flex Repeater of five rows, the validity indicator, the two buttons, and the result label. Owns the custom property <code>result</code> .
RecipeRow	none	One recipe row. Name, active value, staged entry field, and allowed range, each found through the row's parameters. Used only inside the repeater.
Header	shared dock	The existing navigation bar, with one new menu entry for this page.

2.1 How one row finds its tags



The active value follows the same chain through `view.params.active` and reference `a`, but read-only. Unlike the Alarm page, where the row supplies only a name key and the folder word is fixed in the path, here the *whole* tag name is a parameter. That lets two rows use unrelated names and keeps the path in the binding as simply `[default]{s}`.

3 Building the Views

Build in this order: **RecipeRow** first, because the repeater needs it to exist. Save (**Ctrl+S**) at the end of each subsection; an italic view name in the Project Browser means unsaved changes.

Before binding anything, check what is selected

The top of the Property Editor tells you what you are about to change. If it lists **defaultSize**, **dropConfig**, **loading**, or a **PARAMS** section, the *view node* is selected, not a component. A binding added there reads its tag correctly but displays nowhere. Click the component on the canvas first; a Numeric Entry Field shows **value**, **format** and **inputBehavior**, a Label shows **text** and **textStyle**.

3.1 Create the RecipeRow view

Step 1 – New view, flex row

1. **Perspective** → **Views**, right-click → **New View...** Name **RecipeRow**, root **Flex Container**, **Page URL** unchecked.
2. Select root: **direction row**, **alignItems center**, **style.padding 0 12px**, **style.borderRadius 4px**.
3. Select the view node **RecipeRow**: **defaultSize height 40**.

Step 2 – Four view parameters

On the view node, **PARAMS** → **Add View Parameter...** → **Value**, four times. Leave each direction arrow at **Input**.

Name	Default value	Purpose
label	Tank A target	Row name
active	Recipe.TargetA	Tag name of the active value
staged	Staging.TargetA	Tag name of the staged value
range	1-100 %	Allowed-range hint

The defaults only make the row preview in the Designer; the repeater replaces them for every row.

3.2 The four row components

Drop each component directly into **root**, in this order, and set **POSITION** **grow 0** and **shrink 0** on every one. Without **shrink 0**, long names such as *Temp setpoint* wrap onto two lines and the entry field sprouts spinner arrows.

Step 3 – Name label

meta.name Name, **basis 160px**, **props.textStyle.whiteSpace nowrap**. Bind **props.text** with a **Property** binding to **view.params.label** (no braces).

Step 4 – Active label

meta.name Active, basis 60px, props.textStyle.textAlign right, props.style.marginRight 24px. Bind props.text with a **Tag** binding:

Indirect, Tag Path [default]{a}

Reference a: double-click the Property cell, type {view.params.active}, and press **Enter**.

The preview should read 40 (or whatever DS20 holds).

Step 5 – Staged entry field

Drag in a **Numeric Entry Field**: meta.name Staged, basis 90px. Bind props.value with a **Tag** binding:

Indirect, Tag Path [default]{s}

Reference s = {view.params.staged}, committed with **Enter**

Tick **Bidirectional**. This is what makes a typed value write to DS70.

The dialog title must read **Staged.props.value**. If it reads **RecipeRow.props.value**, the binding went onto the view node: copy it (**right-click** → **Copy Binding**), paste it onto the field, and delete the stray value and inputBehavior from the view node.

Step 6 – Range label

meta.name Range, basis 120px, props.style.marginLeft 12px, props.textStyle.color #5F6B76. Bind props.text with a **Property** binding to view.params.range.

Step 7 – Pending-change highlight on the whole row

Select root, add props.style.backgroundColor, and give it an **Expression** binding:

```
if(tag("[default]" + {view.params.staged}) != tag("[default]" + {view.params.active}),
  "#FFF3CD", "transparent")
```

tag() reads a tag whose path is built at run time, which is exactly what a row that only knows tag names needs. With the defaults above (active 40, staged 50) the row turns pale amber on the canvas.

3.3 Create the Recipe view

Step 8 – New view, page URL, custom property

1. **New View...**, name Recipe, root **Flex Container**, tick **Page URL** and enter /recipe.
2. Select root: direction column, alignItems flex-start, style.padding 16px. flex-start lets every item keep its natural width, so buttons and the indicator do not stretch across the page.
3. Select the view node Recipe (the node above root). In **CUSTOM**, click + **Add Custom Property...** → **Value**, rename key to result, and leave the value empty.

A custom property is a variable stored on the view. The Download button writes a message into it; the result label displays it. **Quick test:** type Hello into result once the label from Step 13 exists – the label shows it. Clear it again.

Step 9 – Title

Drag a **Label** into root: `props.text Recipe`, `textStyle.fontSize 24px`, `textStyle.fontWeight bold`, `style.marginBottom 12px`, **POSITION** grow 0 and basis auto. Leaving grow at 1 makes the title swallow half the page.

3.4 The Flex Repeater**Step 10 – Repeater properties**

Drag a **Flex Repeater** into root below the title:

Property	Value	Why
<code>props.path</code>	<code>RecipeRow</code>	The row view
<code>props.direction</code>	<code>column</code>	Rows stack vertically
<code>props.useDefaultViewWidth</code>	<code>false</code>	Rows fit the repeater, not <code>RecipeRow</code> 's 800 px
<code>props.elementPosition</code>	<code>grow 0, shrink 0, basis 40px</code>	Fixed row height
<code>props.style.maxWidth</code>	<code>560px</code>	Amber highlight covers only the row content
<code>position.grow / basis</code>	<code>0 / auto</code>	Repeater takes only the height it needs

Step 11 – Instances from a script transform

1. Click the link icon beside `props.instances` → **Expression**. Enter only the placeholder 1.
2. **Add Transform** + → **Script**. Leave the `def transform(...):` line untouched and replace `return` value with the block below, one indent level in.
3. **Binding Preview** must show **Expression** → 1 and **Script** → an array of 5.

```
return [
    {"label": "Tank A target", "active": "Recipe_TargetA", "staged": "Staging_TargetA", "range": "1-100 %"},
    {"label": "Tank B target", "active": "Recipe_TargetB", "staged": "Staging_TargetB", "range": "1-100 %"},
    {"label": "Mix time", "active": "Recipe_MixTime", "staged": "Staging_MixTime", "range": "1-600 s"},
    {"label": "Temp setpoint", "active": "Recipe_TempSP", "staged": "Staging_TempSP", "range": "70-200 F"},
    {"label": "Recipe number", "active": "Recipe_Number", "staged": "Staging_Number", "range": ""}
]
```

The list goes in the Script transform, never in the Expression box

The Expression box accepts only Ignition's expression language. Pasting the Python `return [...]` there makes the binding fail and can leave the Designer unresponsive until the dialog is cancelled. Keep the range text in plain ASCII (- and F); an en dash or degree sign can raise an encoding error in the script.

3.5 Validity indicator

Step 12 – Label bound to RECIPE_VALID

Drag a **Label** into root below the repeater.

1. **POSITION** grow 0, basis auto.
2. Under `props.style` add `padding 4px 8px`, `borderRadius 4px`, `color #FFFFFF`, `marginTop 8px`.
3. Add `props.style.backgroundColor`: **Tag** → **Direct** `[default]Status_RecipeValid`, **Map** transform with Output Type **Color**: `true` → `#4F8A5B`, `false` → `#D32F2F`, Fallback → `#9E9E9E`.
4. Bind `props.text` with an **Expression**:

```
if({[default]Status_RecipeValid}, "Staged values valid", "Staged values out of range")
```

The indicator is the PLC's own verdict (C112), recomputed every scan, so it changes the moment an out-of-range value is written.

3.6 Result label

Step 13 – Label bound to the custom property

Drag a **Label** into root below where the buttons will go. Bind `props.text` with a **Property** binding to `view.custom.result`. Set `textStyle.fontWeight` bold and `style.marginTop 8px`. Optionally colour it by outcome with an **Expression** on `props.style.color`:

```
if({view.custom.result} = "Accepted", "#2E7D32",
  if(indexOf({view.custom.result}, "Rejected") = 0, "#D32F2F", "#243447"))
```

4 The Two Buttons

Both scripts go in **right-click** → **Configure Events...** → **Component Events** → **onActionPerformed** → **+** → **Script**. The editor shows `def runAction(self, event)::` paste the body under it, one indent level in. Inside the script, `self` is the button and `self.view.custom.result` is the property from Step 8.

Step 14 – Download

Drag a **Button** into root: props.text Download, style.marginTop 12px. Bind props.enabled with an **Expression** so a recipe can only be sent between batches, mirroring the PLC's (DS10=0) condition:

```
{{[default]State} = 0}
```

Do *not* also disable the button for an invalid recipe: sending an invalid recipe is what raises alarm 10, and that is a deliberate test case.

```
def runAction(self, event):
    import time
    staged = ["[default]Staging_TargetA", "[default]Staging_TargetB",
              "[default]Staging_MixTime", "[default]Staging_TempSP", "[default]Staging_Number"]
    active = ["[default]Recipe_TargetA", "[default]Recipe_TargetB",
              "[default]Recipe_MixTime", "[default]Recipe_TempSP", "[default]Recipe_Number"]
    cmd = "[default]Commands_RecipeDownload"

    self.view.custom.result = "Downloading..."
    system.tag.writeBlocking([cmd], [True]) # rising edge -> PLC validates and copies
    time.sleep(1.0) # PLC scan plus tag poll
    system.tag.writeBlocking([cmd], [False]) # release, so the next download can edge again

    s = [q.value for q in system.tag.readBlocking(staged)]
    a = [q.value for q in system.tag.readBlocking(active)]
    if s == a:
        self.view.custom.result = "Accepted"
    else:
        self.view.custom.result = "Rejected - check ranges and the Alarms page"
```

The one-second pause

The script holds the button for about a second so that the PLC has scanned the request and Ignition's tag poll has picked up the new active values before the comparison. If a valid recipe is reported as rejected, the tag group's scan rate is slower than one second: raise the pause to 2.0, or speed up the tag group. The pause blocks only this button's event, not the page.

Step 15 – Copy active to staged

Drag a second **Button** into root: props.text Copy active to staged, style.marginTop 4px. Leave it always enabled – it writes only the staging block.

```
def runAction(self, event):
    staged = ["[default]Staging_TargetA", "[default]Staging_TargetB",
              "[default]Staging_MixTime", "[default]Staging_TempSP", "[default]Staging_Number"]
    active = ["[default]Recipe_TargetA", "[default]Recipe_TargetB",
              "[default]Recipe_MixTime", "[default]Recipe_TempSP", "[default]Recipe_Number"]
    values = [q.value for q in system.tag.readBlocking(active)]
    system.tag.writeBlocking(staged, values)
    self.view.custom.result = ""
```

To place the two buttons side by side, select both, **right-click** → **Wrap in Container** → **Flex Container**, set its direction to row and style.gap to 8px. Wrapping avoids dragging into an existing container.

5 Navigation and the Shared Header

Step 16 – Menu entry and header size

1. Open the **Header** view, select the **Horizontal Menu**, and add a third object to **props.items** with label **Recipe** and **target /recipe**. Check that its own **items** array is **empty**; a menu entry with child items becomes a dropdown instead of a link (the leftover *Link to Google* sample is the usual culprit).
2. Open **Page Configuration**. Confirm **/recipe → Recipe** is listed alongside **/ → Overview** and **/alarms → Alarms**.
3. Click **Shared settings** at the top of the page list, click the pencil on the docked **Header**, and set **Size 50**, **Content push**, **Display visible**, **Resizable** off.
4. Select each page URL in turn. The top dock must show only the **globe** Header (inherited from Shared settings). A **pencil** Header on a page is a page-only copy that overrides the shared one and makes the header height jump between tabs; remove it.

6 Binding Faults and What They Mean

Symptom	Cause and fix
A preview shows the right value but the canvas shows nothing	The binding is on the view node. Move it to the component (Step 5).
A label shows the word <i>Label</i>	Its props.text is not bound yet.
Error_Configuration	An indirect reference cell is empty (edit not committed with Enter) or holds a path without braces.
Bad_NotFound	A tag name in the instances list does not match the Tag Browser. Compare with Copy Path .
Typed value jumps back to the old number	The staging tag is Read Only, or Bidirectional is not ticked.
Nothing writes while testing in the Designer	The Designer's comm mode is read-only. Switch it to Read-/Write on the toolbar, or test in a browser session.
Repeater preview fails; Designer hangs	Python was pasted into the Expression box (Step 11).
Download does nothing visible	The script raised an error – usually a misspelled tag. See Tools → Console , or Status → Logs on the Gateway page; confirm result exists on the <i>view</i> node.
Rows wrap, spinner arrows appear	Row components are allowed to shrink. Set shrink 0 and whiteSpace nowrap (Section 3.2).
Horizontal scrollbar under the rows	useDefaultViewWidth is still true on the repeater.

7 How the Page Is Used

1. **Station idle.** Download is enabled only when **State** reads Idle. Finish or abort the current batch first.

2. **Edit.** Type into a staged field and press Enter or Tab. The value is written to DS70--DS74 immediately and the row turns amber. Nothing in the process changes.
3. **Check.** The indicator must be green. Red means at least one value is outside the range printed beside it.
4. **Download.** The result shows *Downloading...*, then **Accepted**; the active column takes the new values and every amber row clears. The Overview's setpoint and mixing-time labels update, and the next batch uses the new recipe.
5. **Discard.** *Copy active to staged* returns every staged field to the running recipe and clears the result.

Row colour	Indicator	Meaning
plain		Staged equals active. Nothing pending.
amber		Edits pending and acceptable. Download will apply them.
amber		Edits pending, at least one rejected by the PLC's limits. Download will be refused and alarm 10 will seal.

8 Verification Against the Test Criteria

Test action	Expected result
Open <i>Recipe</i> from the header menu	Page opens at /recipe ; five rows; header the same height as on the other pages.
Type 60 into Tank A, press Enter	Staging.TargetA reads 60 in the Tag Browser; the Tank A row turns amber; indicator stays green.
Press Download while idle	Result Accepted in green; Tank A active reads 60; highlight clears; Overview reflects the new recipe.
Press Copy active to staged after editing	All staged fields return to the active values; highlights and result clear.
Type 250 into Temp setpoint	Indicator turns red and reads <i>Staged values out of range</i> .
Press Download with Temp at 250	Result Rejected ; active values unchanged; Alarms page shows alarm 10 sealed and the first-out banner <i>Invalid recipe download</i> .
Restore 160, press Reset on the Alarms page, download again	Alarm 10 clears; download is accepted.
Press Start on the Overview	Download button greys out until the batch returns to Idle.
Download Tank A = 37, then run a batch	Tank A stops at exactly 37 %, confirming the recipe reached the PLC and the level clamp works.
Press Download twice in a row	Both are processed: the script releases C30 each time, so the second press produces a fresh rising edge.

9 Implementation Checks

1. **RecipeRow** has exactly four **Value** parameters, all **Input**, and no page URL. Its view node carries no stray **value** or **inputBehavior** property.
2. Every indirect reference reads `{view.params.active}` or `{view.params.staged}`, with braces, and survives reopening the binding.

3. Only the staged entry field's binding is **Bidirectional**. The active label and every other binding on the page are read-only.
4. All four row components have `shrink 0`; the name label has `whiteSpace nowrap`.
5. The repeater's `instances` binding is an Expression of 1 with a Script transform, previewing an array of five, and `useDefaultViewWidth` is false.
6. The Recipe view node carries the custom property `result`, and the result label is bound to `view.custom.result`.
7. The page writes only `Staging_*` and `Commands_RecipeDownload`. `Recipe_*` (DS20--DS24) are Read Only in the tag provider.
8. The Download script always writes C30 back to `False`; the button is enabled only when `State = 0`.
9. No confirmation logic depends on C31 or C32.
10. The header has one **Recipe** menu entry with an empty `items` array, and every page uses only the shared (globe) docked Header.
11. Later hardening: restrict the Download button to a Supervisor role and enable auditing, so every recipe change is attributable to a user.

Result

One reusable row view and a five-entry list give the operator the running and prepared recipe side by side, a live verdict on the prepared values, and a download that is confirmed by reading the PLC back rather than by chasing one-scan bits – while the PLC remains the only place a recipe is validated and made active.